GUIA COMPLETO

Git & GitHub

Do primeiro commit ao fluxo profissional de desenvolvimento

APOSTILA PRÁTICA

Fundamentos • comandos • branches • colaboração • segurança • recuperação de erros • GitHub Actions

Edição de estudo • Outubro de 2026

Material organizado para leitura progressiva, consulta rápida e prática no terminal. Os comandos são exemplos: substitua nomes, caminhos e URLs pelos do seu projeto.

Como usar esta apostila

Este guia foi estruturado para quem está começando, mas também inclui ferramentas e práticas utilizadas em equipes de desenvolvimento. Estude na sequência e execute os exemplos em um repositório de teste antes de aplicar comandos de recuperação em projetos importantes.

Regra de ouro: antes de executar um comando que altera o histórico, apaga arquivos ou reescreve commits, rode `git status` e confirme o que será afetado.

Mapa de conteúdo

01 Fundamentos: Git, GitHub e controle de versão
02 Instalação e configuração
03 O ciclo de trabalho e os comandos essenciais
04 Repositórios locais e remotos
05 Branches, merge, rebase e conflitos
06 GitHub na prática: PRs, Issues, README e colaboração
07 Autenticação: HTTPS, tokens e SSH
08 Segurança, .gitignore e segredos
09 Desfazer e recuperar alterações
10 Fluxos profissionais, qualidade e automação
11 Erros frequentes e diagnóstico
12 Projeto guiado, exercícios e checklist
13 Glossário e referências oficiais

01. Fundamentos

O que é controle de versão?

Controle de versão registra mudanças em arquivos ao longo do tempo. Com ele, é possível entender o que mudou, comparar versões, recuperar estados anteriores e desenvolver funcionalidades em paralelo sem precisar criar cópias manuais como projeto-final-2 ou projeto-final-definitivo.

Git e GitHub não são a mesma coisa

Termo	O que significa	Exemplo de uso
Git	Sistema distribuído de controle de versão, executado localmente ou em servidores.	Criar commits, branches e comparar versões.
GitHub	Plataforma que hospeda repositórios Git e oferece colaboração, revisão, Issues e automações.	Compartilhar um portfólio e revisar um pull request.
Repositório local	Repositório Git que está no seu computador.	Pasta do projeto com a pasta oculta .git.
Repositório remoto	Outra cópia do repositório acessível por rede.	Repositório do projeto no GitHub.

Modelo mental: onde as alterações ficam

Área	Papel	Comando associado
Working tree	Arquivos como estão no disco.	git status, git diff
Staging area / index	Seleção exata que entrará no próximo commit.	git add, git diff --staged
Histórico local	Commits registrados no repositório.	git commit, git log
Remoto	Cópia hospedada, por exemplo, no GitHub.	git fetch, git pull, git push

Fluxo típico: editar → revisar → `git add` → `git commit` → `git push`. O commit é local; o push envia os commits ao remoto.

Vocabulário essencial

Termo	Definição curta
Commit	Registro de um conjunto de alterações, identificado por um hash.
Branch	Referência móvel para uma linha de desenvolvimento.
Merge	Integra históricos de branches.
Rebase	Reaplica commits sobre outra base, reescrevendo o histórico reaplicado.
Clone	Copia um repositório remoto para o computador.
Fetch	Busca referências e objetos do remoto sem integrar automaticamente na branch atual.
Pull	Busca alterações e integra-as à branch atual (merge ou rebase conforme configuração).
Push	Envia commits locais para um remoto.
Pull request (PR)	Proposta de revisar e integrar alterações no GitHub.
HEAD	Referência à posição atual do checkout, geralmente o commit da branch atual.
Origin	Nome convencional do remoto principal; não é uma palavra especial obrigatória.
Conflito	Alterações que o Git não consegue combinar automaticamente.

02. Instalação e configuração

Instalar Git

- Windows: https://git-scm.com/downloads. Durante a instalação, as opções padrão costumam ser adequadas para iniciantes.
- Ubuntu/Debian: execute os comandos abaixo.

```
sudo apt update
sudo apt install git
```

- macOS: instale as ferramentas de linha de comando do Xcode ou utilize um gerenciador de pacotes.

```
git --version
```

Se o terminal mostrar uma versão, o Git está disponível. No Windows, você pode usar Git Bash, PowerShell ou o terminal integrado do VS Code.

Configurar identidade

```
git config --global user.name "Seu Nome"
git config --global user.email "seu-email@example.com"
git config --list
```

O nome e o e-mail ficam registrados nos metadados dos commits; eles não são a senha do GitHub. Use um endereço que queira associar à sua atividade ou configure o e-mail privado fornecido pelo GitHub.

Configurações úteis

```
git config --global init.defaultBranch main
# Opcional: usar VS Code como editor, se "code" estiver no PATH
git config --global core.editor "code --wait"
```

Para um único projeto, entre na pasta e omita `--global` para configurar nome ou e-mail apenas naquele repositório.

Checar a configuração efetiva

```
git config --show-origin --get-regexp 'user\.|init.defaultBranch|core.editor'
git status
```

Não configure senhas, tokens ou chaves privadas no nome, e-mail, URL de remoto ou em arquivos versionados. Credenciais devem ser gerenciadas por métodos próprios de autenticação.

03. O ciclo de trabalho e comandos essenciais

Criar ou inspecionar um repositório

```
git init
git status
git branch --show-current
git log --oneline --decorate
```

`git init` inicializa um repositório na pasta atual. Evite executá-lo acidentalmente em uma pasta-pai que contenha vários projetos.

Adicionar, revisar e registrar

```
git status
git add README.md
git diff --staged
git commit -m "Adiciona documentação inicial"
```

`git add .` prepara as alterações encontradas no diretório atual e abaixo dele. Antes de usar em projetos grandes, confira o status para não incluir arquivos indevidos.

Entender as diferenças

```
git diff                # alterações ainda fora do staging
git diff --staged       # alterações preparadas
git diff HEAD           # diferenças entre a árvore de trabalho e HEAD
```

Mensagens de commit

Uma mensagem boa descreve a intenção da alteração. Prefira uma mensagem curta, específica e escrita no imperativo ou no estilo adotado pela equipe.

Fraca	Melhor
mudanças	Corrige validação de e-mail
update	Adiciona busca de livros por título
fix	Impede envio de formulário vazio
coisas novas	Cria endpoint de autenticação

Exemplo completo

```
git status
# Edite e teste os arquivos
git status
git diff
git add src/login.js
git diff --staged
git commit -m "Valida credenciais no login"
git status
```

O commit não envia nada ao GitHub. Para compartilhar commits, use `git push` depois de configurar o remoto.

04. Repositórios locais e remotos

Clonar um projeto

```
git clone https://github.com/USUARIO/REPOSITORIO.git
cd REPOSITORIO
git remote -v
```

Clonar cria uma cópia local do repositório, incluindo o histórico disponível e a configuração de remoto. Substitua a URL de exemplo pela URL real.

Conectar projeto local existente

```
git remote add origin https://github.com/USUARIO/REPOSITORIO.git
git branch -M main
git push -u origin main
```

Use esta sequência apenas quando o projeto já tiver um commit local e o remoto estiver preparado para recebê-lo. Se o remoto já contiver commits iniciais independentes, como um README criado no site, pode ser necessário integrar os históricos antes do push.

Comandos de sincronização

Comando	Efeito	Quando usar
git remote -v	Lista URLs e nomes dos remotos.	Verificar para onde o push será enviado.
git fetch origin	Atualiza referências remotas locais.	Inspecionar novidades antes de integrar.
git pull	Busca e integra alterações.	Atualizar a branch atual.
git push	Envia commits para o remoto configurado.	Publicar commits.
git push -u origin nome-da-branch	Envia a branch e configura acompanhamento.	Primeiro push de uma branch.
git branch -vv	Mostra branches e upstreams.	Diagnosticar acompanhamento remoto.

Público ou privado?

- Público: qualquer pessoa pode visualizar o conteúdo. É útil para portfólio, exemplos e projetos open source.
- Privado: o acesso é restrito às pessoas autorizadas, mas isso não elimina a necessidade de proteger segredos.
- Nunca publique código de cliente, dados pessoais, credenciais ou material sem autorização.

05. Branches, merge, rebase e conflitos

Criar e trocar de branch

```
git branch                          # lista branches locais
git switch -c feat/busca-livros     # cria e troca
git switch main                     # volta para main
```

Nomes descritivos ajudam a equipe a entender a finalidade da branch. Prefixos comuns incluem `feat/`, `fix/`, `docs/` e `refactor/`; não são obrigatórios.

Integrar com merge

```
git switch main
git pull
git merge feat/busca-livros
```

O merge integra os commits da branch de origem. Dependendo do histórico, pode avançar a referência diretamente (fast-forward) ou criar um commit de merge.

Rebase: o que muda?

```
git switch feat/busca-livros
git fetch origin
git rebase origin/main
```

O rebase reaplica commits da branch sobre uma base mais recente, produzindo novos hashes. Pode deixar o histórico linear, mas exige cuidado se outros já dependem dos commits antigos.

Regra segura: evite fazer rebase de commits compartilhados/publicados sem combinar com a equipe. Nunca force push em uma branch compartilhada sem entender as consequências.

Resolver conflitos

Quando o Git não consegue decidir como combinar alterações, ele marca o arquivo com delimitadores semelhantes a estes:

```
<<<<<<< HEAD
versão atual
=======
versão que está sendo integrada
>>>>>>> outra-branch
```

- Abra o arquivo e decida qual conteúdo deve permanecer.
- Remova os marcadores e confira o resultado completo.
- Execute testes e verifique outros arquivos afetados.
- Marque o conflito como resolvido com `git add arquivo`.
- Finalize o merge com `git commit` se o Git solicitar, ou continue o rebase com `git rebase --continue`.

Para desistir de uma operação em andamento, use o comando de aborto correspondente somente depois de conferir o estado: `git merge --abort` ou `git rebase --abort`.

Fluxo recomendado para uma funcionalidade

```
git switch main
git pull
git switch -c feat/paginacao
# Desenvolva, teste e faça commits
git add .
git commit -m "Adiciona paginação à lista de livros"
git push -u origin feat/paginacao
```

Depois, abra um pull request no GitHub, peça revisão e integre a branch conforme as regras do projeto.

06. GitHub na prática

Pull requests (PRs)

Um pull request propõe integrar alterações de uma branch em outra. Ele é o local para explicar a mudança, discutir o código, observar testes automáticos e registrar a revisão.

- Use um título específico e uma descrição que explique problema, solução, testes e impactos.
- Mantenha PRs pequenos o bastante para serem revisados.
- Vincule Issues relacionadas quando fizer sentido.
- Responda a comentários e atualize a branch até cumprir as verificações exigidas.

Issues e Projects

Issues organizam bugs, melhorias, dúvidas e tarefas. GitHub Projects ajuda a acompanhar o trabalho em quadros ou tabelas. Em projetos individuais, uma Issue por funcionalidade também pode ajudar a planejar o portfólio.

README de qualidade

O README é a página de entrada do repositório. Deve permitir que outra pessoa entenda o projeto e consiga executá-lo.

- Nome e descrição objetiva do projeto.
- Problema que resolve e principais funcionalidades.
- Tecnologias utilizadas.
- Pré-requisitos e instruções de instalação.
- Como configurar variáveis de ambiente, com um arquivo de exemplo sem segredos.
- Como executar e testar.
- Capturas de tela ou demonstração, quando úteis.
- Limitações conhecidas, licença e formas de contribuição.

Modelo de README

```
# Nome do projeto

Descrição curta do problema que ele resolve.

## Funcionalidades
- Funcionalidade 1
- Funcionalidade 2

## Tecnologias
- Linguagem / framework
- Banco de dados

## Requisitos
- Versão da linguagem
- Dependências necessárias

## Instalação
1. Clone o repositório.
2. Instale as dependências.
3. Configure o ambiente usando `.env.example`.
4. Execute os comandos de inicialização.

## Testes
Explique como executar os testes.

## Licença
Informe a licença, se aplicável.
```

Perfil e portfólio

- Fixe os projetos mais representativos no perfil.
- Use descrições, tópicos (topics), README claro e histórico compreensível.
- Inclua projetos executáveis ou demonstrações quando possível.
- Não infle o portfólio com cópias de tutoriais sem explicar o que você aprendeu ou modificou.
- Mantenha o código e as instruções atualizados.

07. Autenticação: HTTPS, tokens e SSH

HTTPS

O GitHub não aceita mais autenticação de operações Git por senha tradicional da conta. Para operações autenticadas via HTTPS, use um gerenciador de credenciais compatível ou um token de acesso pessoal (PAT), conforme o método escolhido.

SSH

SSH permite autenticar com um par de chaves. A chave pública pode ser cadastrada no GitHub; a chave privada deve permanecer protegida no seu dispositivo.

```
ssh-keygen -t ed25519 -C "seu-email@example.com"
# Depois, adicione a chave pública ao GitHub conforme a documentação
ssh -T git@github.com
```

O comando de geração pode perguntar onde salvar a chave e se deseja usar uma frase secreta (passphrase). Para projetos de trabalho, siga a política de segurança da organização.

HTTPS ou SSH?

Método	Vantagens	Atenção
HTTPS	Fácil de usar em muitos ambientes; pode funcionar com gerenciador de credenciais.	Não cole tokens em URLs, comandos compartilhados ou arquivos.
SSH	Autenticação conveniente após configuração; útil em uso frequente.	Proteja a chave privada e revogue chaves antigas ou comprometidas.

Se um token ou chave privada for exposto, revogue-o imediatamente e gere uma nova credencial. Apagar o texto do arquivo não remove o segredo do histórico Git.

08. Segurança, .gitignore e segredos

O que não deve ser versionado?

- Senhas, tokens, chaves privadas, credenciais de banco e arquivos de configuração reais.
- Pastas de dependências e artefatos gerados, quando não devem fazer parte do projeto (por exemplo, node_modules).
- Arquivos temporários, logs locais, cache e configurações específicas da máquina.
- Dados de clientes, bases reais e arquivos com informações pessoais sem autorização.

Exemplo de .gitignore

```
# Dependências
node_modules/

# Segredos e configurações locais
.env
.env.*
!.env.example

# Logs
*.log

# Saídas de build
build/
dist/

# Sistema operacional
.DS_Store
Thumbs.db

# Editores (ajuste conforme a equipe)
.vscode/
```

Adapte o arquivo ao projeto. Não ignore automaticamente arquivos necessários à construção, testes ou colaboração. O padrão `.env.*` também ignora arquivos como `.env.example`; a exceção mostrada acima permite versionar o modelo sem credenciais.

Variáveis de ambiente

Guarde valores sensíveis no ambiente de execução ou em um gerenciador de segredos. Versione apenas um arquivo de exemplo com nomes de variáveis e valores fictícios.

```
# .env.example — valores ilustrativos, sem segredos reais
DATABASE_URL=postgres://usuario:senha@localhost:5432/banco_exemplo
JWT_SECRET=troque_por_um_valor_local
```

Se um arquivo já foi commitado

Adicionar o arquivo ao .gitignore não remove versões anteriores do histórico. Se o segredo foi publicado, revogue-o primeiro. Depois, remova o arquivo do índice sem apagar a cópia local, quando apropriado:

```
git rm --cached .env
git add .gitignore
git commit -m "Remove arquivo de ambiente do controle de versão"
```

Esse procedimento não apaga o segredo de commits antigos. Em caso de exposição, a prioridade é invalidar a credencial. A limpeza do histórico pode ser necessária, mas deve ser planejada, pois altera hashes e exige coordenação com quem clonou o repositório.

Princípios de segurança

- Use o menor nível de acesso necessário.
- Ative autenticação multifator na conta GitHub.
- Revise permissões de Actions e aplicativos de terceiros.
- Não execute scripts de repositórios desconhecidos sem entender o que fazem.
- Use análise de dependências e proteção de segredos quando disponível.
- Em projetos profissionais, siga políticas internas e requisitos legais aplicáveis.

09. Desfazer e recuperar alterações

Atenção: comandos de recuperação podem descartar trabalho. Leia o estado do repositório e, se houver dúvida, crie uma cópia ou branch de segurança antes.

Descartar alterações não commitadas

```
git diff
# Descartar alterações em um arquivo específico:
git restore caminho/arquivo.js
```

Isso descarta modificações não preparadas naquele arquivo. Não use se precisar preservar essas alterações.

Retirar um arquivo do staging

```
git restore --staged caminho/arquivo.js
```

O arquivo continua modificado na pasta de trabalho; apenas deixa de estar preparado para o commit.

Corrigir o último commit

```
git add arquivo-corrigido.js
git commit --amend
```

O amend substitui o commit anterior por outro. Se o commit já foi publicado, o histórico muda; combine com a equipe antes de atualizar o remoto.

Reverter um commit publicado

```
git log --oneline
git revert HASH_DO_COMMIT
```

O revert cria um novo commit que desfaz os efeitos de outro commit. Em geral, é uma opção mais segura para desfazer alterações já compartilhadas.

Reset: entender antes de usar

Comando	Efeito geral	Cuidado
git reset --soft HEAD~1	Move HEAD; mantém mudanças preparadas.	Reescreve o histórico local.
git reset --mixed HEAD~1	Move HEAD; mantém mudanças no diretório, fora do staging.	É o modo padrão do reset.
git reset --hard HEAD~1	Move HEAD e descarta alterações rastreadas do estado de trabalho/index.	Pode destruir trabalho não salvo.

Recuperar referência com reflog

```
git reflog
```

O reflog registra movimentos recentes de referências locais, o que pode ajudar a localizar um commit que deixou de aparecer na branch. Os registros não são permanentes e não substituem backups.

Arquivos deletados ou histórico confuso

- Pare e execute `git status` e `git log --oneline --graph --decorate --all`.
- Não execute repetidamente comandos destrutivos tentando adivinhar a solução.
- Se o trabalho for valioso, faça uma cópia da pasta antes de tentar recuperação.
- Para conflitos, resolva os arquivos e finalize a operação em andamento; não misture comandos de merge e rebase sem identificar qual operação está ativa.

10. Fluxos profissionais, qualidade e automação

Um fluxo simples de equipe

```
git switch main
git pull
git switch -c fix/erro-validacao
# Faça a correção e os testes
git add src/validacao.js
git commit -m "Corrige validação de entrada"
git push -u origin fix/erro-validacao
```

Abra um PR, descreva a mudança, solicite revisão, confira testes e integre a branch segundo as regras do projeto. Depois da integração, atualize a branch principal local.

Git Flow ou trunk-based?

Modelo	Como funciona	Quando considerar
Branch curta + PR	Cada tarefa tem branch curta e revisão por pull request.	Boa opção geral para equipes pequenas e médias.
Trunk-based development	Integrações frequentes em uma branch principal, com branches muito curtas e testes robustos.	Equipes que entregam continuamente e têm automação madura.
Git Flow	Branches com papéis mais definidos, como develop, release e hotfix.	Produtos com ciclos de release específicos; pode ser excessivo para projetos simples.

Qualidade de commits

- Faça commits que representem uma mudança lógica.
- Evite incluir arquivos gerados, segredos ou formatação não relacionada.
- Execute testes antes de abrir um PR.
- Mantenha as mudanças pequenas e a descrição útil.
- Não reescreva histórico compartilhado sem acordo.

GitHub Actions

GitHub Actions automatiza tarefas acionadas por eventos, como push e pull request. Pode executar testes, lint, build e verificações de qualidade. Para usar em produção, configure permissões mínimas, proteja segredos e revise workflows de terceiros.

```
# Exemplo introdutório: .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Exemplo de etapa
        run: echo "Configure aqui os comandos de teste do projeto"
```

O exemplo é ilustrativo. Em workflows reais, use versões de Actions adequadas à política de segurança do projeto e substitua o comando demonstrativo por testes reais.

Licenças e open source

Antes de publicar um projeto, verifique se você tem direito de compartilhar o código e suas dependências. Um repositório público não significa automaticamente que outras pessoas têm permissão irrestrita para reutilizar o conteúdo. Escolha uma licença adequada e siga as condições das bibliotecas utilizadas.

11. Erros frequentes e diagnóstico

Mensagem/situação	Causa provável	Primeiros passos
Author identity unknown	Nome/e-mail não configurados.	Configure user.name e user.email.
Permission denied (publickey)	Chave SSH ausente, não cadastrada ou não carregada.	Verifique a chave e teste ssh -T.
Authentication failed	Credencial HTTPS inválida ou expirada.	Use o gerenciador de credenciais ou atualize o método de autenticação.
remote origin already exists	Já existe um remoto com esse nome.	Use git remote -v; ajuste com git remote set-url origin URL se for correto.
rejected / non-fast-forward	O remoto tem commits que sua branch ainda não integrou.	Faça fetch, inspecione e integre as mudanças.
nothing to commit	Não há alterações preparadas ou arquivos modificados rastreados.	Confira git status e se os arquivos estão ignorados.
merge conflict	Alterações incompatíveis no mesmo contexto.	Edite os arquivos, remova marcadores, teste e finalize.
file is too large	Arquivo excede limites ou não deveria estar no Git.	Considere remover o artefato ou usar Git LFS quando apropriado.
.env apareceu no GitHub	O arquivo foi rastreado ou commitado antes do ignore.	Revogue segredos expostos; remova do índice e planeje limpeza do histórico.

Checklist de diagnóstico

```
git status
git branch --show-current
git remote -v
git log --oneline --decorate -8
git diff
git diff --staged
```

Esses comandos são de inspeção e ajudam a entender o estado atual antes de fazer mudanças. Se houver uma operação em andamento, identifique-a antes de tentar abortar ou continuar.

12. Projeto guiado e exercícios

Projeto prático: lista de tarefas

Objetivo: criar um pequeno repositório com um README, um arquivo de código de exemplo, commits claros, uma branch de funcionalidade e um pull request.

Etapa 1 — iniciar

```
mkdir lista-de-tarefas
cd lista-de-tarefas
git init
git branch -M main
```

Etapa 2 — documentar

```
# Lista de tarefas

Projeto de estudo para praticar Git e GitHub.

## Funcionalidades planejadas
- Adicionar tarefa
- Listar tarefas
- Marcar tarefa como concluída
```

Salve esse conteúdo em `README.md`, depois:

```
git add README.md
git commit -m "Adiciona README inicial"
```

Etapa 3 — criar uma branch

```
git switch -c feat/listar-tarefas
```

Crie um arquivo simples da linguagem que estiver estudando, por exemplo `tarefas.py`, com uma lista de tarefas e uma função que as imprima. Teste o código.

```
git add tarefas.py
git commit -m "Implementa listagem de tarefas"
```

Etapa 4 — publicar

Crie um repositório vazio no GitHub, conecte-o usando a URL correta e envie a branch principal e a branch de funcionalidade. Abra um PR de `feat/listar-tarefas` para `main`.

```
git switch main
git push -u origin main
git push -u origin feat/listar-tarefas
```

Se o remoto não tiver histórico próprio e a branch principal já tiver commit local, o push deve publicar os commits. Caso o remoto tenha sido inicializado com arquivos ou commits diferentes, inspecione e integre os históricos em vez de forçar o envio.

Exercícios de fixação

- Explique com suas palavras a diferença entre Git e GitHub.
- Qual é a diferença entre git add, git commit e git push?
- Como listar os remotos configurados?
- Como criar e trocar para uma branch nova?
- Quando você escolheria git revert em vez de git reset?
- Por que adicionar .env ao .gitignore não resolve sozinho um segredo já publicado?
- O que fazer quando git push é rejeitado por non-fast-forward?
- Quais informações devem constar em um bom README?
- O que é um pull request e por que ele ajuda equipes?
- Como verificar alterações antes de registrá-las?

Respostas resumidas

1. Git controla versões; GitHub hospeda repositórios Git e oferece colaboração. 2. add prepara, commit registra localmente, push envia ao remoto. 3. `git remote -v`. 4. `git switch -c nome-da-branch`. 5. Revert costuma ser preferível para desfazer alterações compartilhadas, pois cria um novo commit. 6. O segredo continua em commits anteriores e precisa ser revogado. 7. Faça fetch, inspecione e integre as mudanças do remoto. 8. Objetivo, funcionalidades, instalação, configuração, execução, testes e licença. 9. É uma proposta de integração com revisão e verificações. 10. Use status, diff e diff --staged.

13. Checklist de domínio

Marque cada item quando conseguir executá-lo sem depender de um tutorial.

- ■ Instalar Git e verificar a versão.
- ■ Configurar nome e e-mail.
- ■ Criar repositório local e consultar git status.
- ■ Preparar arquivos com git add e revisar com git diff --staged.
- ■ Criar commits com mensagens claras.
- ■ Clonar um repositório e identificar o remoto.
- ■ Enviar e receber alterações com push, fetch e pull.
- ■ Criar branch, fazer merge e compreender rebase.
- ■ Resolver um conflito de merge em um projeto de teste.
- ■ Abrir um pull request e responder a uma revisão.
- ■ Criar README e .gitignore apropriados.
- ■ Evitar versionar credenciais e saber como reagir a um segredo exposto.
- ■ Escolher entre restore, revert e reset conforme o caso.
- ■ Ler o histórico com log e localizar referências com reflog.
- ■ Explicar o fluxo de trabalho do projeto para outra pessoa.

Glossário rápido

Comando	Para lembrar
git status	Estado atual da árvore de trabalho e do staging.
git add	Prepara alterações para o próximo commit.
git commit	Registra as alterações preparadas.
git log --oneline	Mostra um histórico compacto.
git diff	Mostra diferenças não preparadas.
git diff --staged	Mostra diferenças preparadas.
git switch -c nome	Cria e troca para uma branch.
git merge nome	Integra a branch indicada na branch atual.
git fetch	Busca referências e objetos remotos.
git pull	Busca e integra alterações remotas.
git push	Envia commits ao remoto.
git restore arquivo	Descarta alterações não preparadas no arquivo.
git restore --staged arquivo	Retira o arquivo do staging.
git revert HASH	Cria commit que desfaz outro commit.
git reflog	Consulta movimentos recentes das referências locais.

Referências oficiais e continuidade

Use a documentação oficial para conferir detalhes que variam conforme versão, configuração ou política de uma organização:

- Git — documentação: https://git-scm.com/doc
- Git Book, gratuito: https://git-scm.com/book/pt-br/v2
- GitHub Docs: https://docs.github.com/
- GitHub Skills — exercícios práticos: https://skills.github.com/
- GitHub sobre autenticação por SSH: https://docs.github.com/en/authentication/connecting-to-github-with-ssh
- GitHub sobre autenticação via HTTPS: https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/about-authentication-to-github
- GitHub Actions: https://docs.github.com/en/actions

A documentação oficial deve ser a referência final para sintaxe, autenticação, limites, segurança e recursos atuais. Este guia é um material de estudo e não substitui as políticas específicas de um repositório ou organização.